The Journal of Open Source Software

notata: Structured Filesystem Logging for Scientific Runs

Albert Alonso [1]

1 Aalborg University, Aalborg, Denmark

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 26 January 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

notata is a lightweight Python library designed to log scientific simulations and experiments directly to the filesystem. It provides a minimal API to organize simulation outputs, such as parameters, numerical arrays, plots, and metadata, into a structured, reproducible directory hierarchy. Unlike machine learning platforms, notata targets scientific workflows where filesystem accessibility, local reproducibility, and integration with standard Unix tools (like grep and find) are the priority. It offers a standardized scaffolding for managing results without requiring additional dependencies.

Statement of Need

Scientific computing workflows often involve solving differential equations, stochastic simulations, or parameter sweeps. These processes generate heterogeneous outputs, including binary arrays, diagnostic plots, and configuration files, which must be organized for future retrieval and analysis.

Most existing logging solutions, such as Weights & Biases (Biewald, 2020), TensorBoard (Wonka et al., 2015), MLflow (Zaharia et al., 2020), were designed for machine learning model training in mind, where the primary concern is real-time monitoring, hyperparameter tracking, and cloud-based dashboards. These tools impose significant infrastructure overhead (servers, databases, APIs) and are optimized for scenarios with frequent scalar metric writes rather than heterogeneous data types or reproducibility-first workflows.

Scientific computing research, by contrast, often operates in more ad-hoc scenarios, where each researcher stores and logs their work in a different manner. For a field that values reproducibility and audit trails as primary concerns, the current approach results in less than an ideal solution. Thus, it can clearly benefit from results that are easy to explore, directly queryable via Unix tools and version-controllable, and remove the need for the user to place all the scaffolding needed. Practically, a researcher running simulations might need to:

- Compare results across 100+ parameter combinations, querying results via grep or find.
- Reproduce a simulation months later by inspecting saved parameters.
- Version-control results alongside code using standard Git workflows.
- Run simulations on machines without internet connectivity or cloud access.
- Integrate logging with shell scripts and existing computational pipelines.
- Explore previous work's results to reproduce figures.

Current tools fall short because they either: require external infrastructure, optimize for real-time dashboards over filesystem accessibility, or impose schemas that don't align with scientific computing practices.

notata addresses this gap by placing filesystem-level reproducibility, accessibility, and portability

at the center of its design. The library captures domain expertise in managing parameter–result associations through a simple, zero-infrastructure API that integrates naturally with Unix tools and version control systems.

State of the field

Existing approaches to experiment logging and result management fall into three broad categories.

General-purpose logging frameworks, such as Python's built-in `logging` module and operational monitoring stacks like the ELK stack (Elastic, 2024), are designed for software observability rather than scientific reproducibility. They focus on streaming messages and system metrics and provide limited support for organizing heterogeneous scientific outputs or associating results with experimental parameters.

Experiment tracking platforms developed for machine learning, such as Weights & Biases (Biewald, 2020), TensorBoard (Wonka et al., 2015), and MLflow (Zaharia et al., 2020), prioritize real-time visualization, cloud-backed storage, and hyperparameter optimization. These tools assume frequent scalar logging and persistent services, making them poorly suited for batch-oriented scientific simulations, offline environments, or workflows where the filesystem is the primary interface to results.

Workflow management systems, including Snakemake (Köster & Rahmann, 2012), address execution and dependency tracking rather than run-level result organization. While they structure how computations are executed, they do not prescribe how outputs from individual runs should be stored, inspected, or compared over time. Domain-specific tools such as Sacred (Greff et al., 2017) provide experiment management abstractions but are closely tied to specific workflow patterns and often assume database-backed metadata storage.

In contrast, `notata` focuses exclusively on the organization of scientific run outputs on disk. Rather than introducing new services or execution models, it formalizes a common ad-hoc practice by enforcing a consistent filesystem structure and metadata conventions. This narrow scope addresses a gap not covered by existing tools, reproducible filesystem-native logging for scientific simulations that must remain accessible independently of external infrastructure.

Design and implementation

Core Design Tradeoffs

The architecture of `notata` is driven by the specific constraints of scientific workflows, prioritizing long-term reproducibility and ease of deployment over the real-time monitoring features found in machine learning tools.

- **Filesystem-centric persistence**: Scientific simulations frequently run in environments where deploying databases or external logging services is impractical, such as HPC clusters or restricted networks. `notata` relies exclusively on the local filesystem as its data store. This approach ensures that results are intrinsically portable and remain accessible via standard Unix utilities (like `ls`, `grep`, and `find`) independent of the library itself.
- **Opinionated structure**: Rather than offering a free-form logging facility, `notata` enforces a standardized directory schema tailored to scientific outputs. Parameters are strictly separated from results, with metadata stored in `YAML` and numerical data in `NumPy` formats. This standardization decouples data generation from analysis: because the output structure is guaranteed, researchers can write generic post-processing tools that do not need to understand the internal logic of the simulation code.
- **Minimalism and maintainability**: The library intentionally limits its scope to the essential artifacts of research: parameters, arrays, logs, and plots. By avoiding feature bloat, it

DRAFT

remains lightweight and easy to audit. This minimal API surface ensures the library is stable and can be easily integrated into existing codebases without introducing heavy dependencies.

Directory Structure and Reproducibility

Each run generates a self-contained, timestamped directory with clear separation of concerns:

```
log_<run_id>/
├── params.yml          # Parameters (human-readable, version-controllable)
├── metadata.yml        # System info, timestamps, software versions
├── log.txt             # Timestamped messages and progress
├── arrays/             # Numerical arrays (NumPy .npz/.npy format)
│   ├── trajectory.npz
│   └── statistics.npz
└── plots/              # Visualizations (PNG/PDF)
    ├── phase_space.png
    └── energy.png
```

This structure supports:

- **Querying:** Users can use `grep` to filter runs by parameter or outcome (e.g., `grep "diverged" */log.txt`).
- **Version Control:** Text-based parameter files are git-friendly.
- **Auditability:** Metadata files automatically capture the execution environment.

Typical Workflows

Example 1: Parameter Sweep with Result Analysis

A researcher running a parameter sweep over a set of configurations can store each run's results and later analyze them via shell:

```python
from notata import Experiment
import numpy as np

# Define parameter sweep
params_list = [
  {'omega': omega, 'dt': dt}
  for omega in [1.0, 2.0, 3.0]
  for dt in [1e-3, 1e-4, 1e-5]
]

exp = Experiment(name="parameter_sweep", params_list=params_list)
for log in exp:
  omega, dt = log.params['omega'], log.params['dt']
  energy = simulate(omega, dt)

  log.arrays('energy', energy=energy)
  if np.all(energy < threshold):
    log.info("stable")
  else:
    log.info("unstable")
```

Because the output is standard, analysis can be performed via shell:

```shell
grep "Status: stable" parameter_sweep_*/log_*/log.txt
```

DRAFT

Or aggregated back into Python:

```python
from notata import ExperimentReader

reader = ExperimentReader('parameter_sweep')
results = [
    (run.params['omega'], run.arrays('energy')) for run in reader.runs
]
```

Example 2: Long-Running Simulation with Checkpointing

A multi-hour simulation can log progress and save checkpoints:

```python
with Logbook("long_sim", params=config, autosave_dir="checkpoints") as log:
  for epoch in range(1000):
    state = compute_epoch(state)
    if epoch % 100 == 0:
      log.info(f"Epoch {epoch}: loss={state.loss}")
      log.arrays(f"checkpoint_{epoch}", state=state)
```

Results are immediately queryable and version-controllable, enabling reproducibility and post-hoc analysis without external services.

Research Impact Statement

notata is released as an open-source Python package with automated tests, versioned documentation, and reproducible examples. It is actively used to manage numerical simulation outputs and parameter sweeps in ongoing research workflows.

The software integrates directly into existing scientific Python codebases without external services or infrastructure, making it suitable for HPC and offline environments. Its fixed output structure enables reuse of analysis scripts across projects and supports reproducible figure generation from archived runs.

Future steps

Future development will focus on incremental extensions driven by user needs, including support for additional on-disk data formats and tighter integration with existing scientific Python tooling.

Availability

notata is distributed via PyPI (`pip install notata`) and the source code is available at https://github.com/alonfnt/notata under the MIT License. Complete documentation, including tutorials, API reference, and design rationale, is available at https://notata.readthedocs.io.

AI Usage Disclosure

Generative AI tools were used for limited assistance with documentation editing and manuscript organization. All software design decisions, implementation, examples, and technical claims were produced and verified by the authors.

DRAFT

References

Biewald, L. (2020). *Weights & biases: Machine learning experiment tracking*. https://wandb.ai/

Elastic. (2024). *Elastic stack: Search, observe, and protect*. https://www.elastic.co/elastic-stack

Greff, K., Klein, A., Chrabaszcz, M., Biewer, F., Rebhan, U., & Thesen, F. (2017). Sacred: A python library for easy access to experiment meta-information. *German Conference on Artificial Intelligence*, 265–279.

Köster, J., & Rahmann, S. (2012). Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, *28*(19), 2520–2522. https://doi.org/10.1093/bioinformatics/bts480

Wonka, D., Lieder, F., & others. (2015). *TensorBoard: TensorFlow's visualization toolkit*. https://www.tensorflow.org/tensorboard

Zaharia, M., Chen, A., Davidson, A., & others. (2020). *MLflow: A machine learning lifecycle platform*. https://mlflow.org/

DRAFT